

[Home](#)[GitHub](#)

CLASSES

[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)[Footer](#)[GET /](#)

frontend/src/pages/Seguimiento.jsx

```
1.  /**
2.   * @file Página de Seguimiento Diario.
3.   * @description Un formulario de varios pasos para que los usuarios
4.   * @requires react
5.   * @requires react-router-dom
6.   * @requires ../store/authStore
7.   * @requires ../config/api
8.   * @requires ../components/molecules/EmotionSelector
9.   * @requires ../components/molecules/ActivitySelector
10.  * @requires ../components/molecules/CognitionSelector
11.  * @requires ../components/atoms/Slider
12.  * @requires ../components/atoms/Checkbox
13.  * @requires ../components/atoms/Button
14.  * @requires ../styles/Seguimiento.css
15.  */
16. import React, { useState } from "react";
17. import { useNavigate } from "react-router-dom";
18. import { useAuthStore } from "../store/authStore";
19. import { apiConfig } from "../config/api";
20. import EmotionSelector from "../components/molecules/EmotionSelector";
21. import ActivitySelector from "../components/molecules/ActivitySelector";
22. import CognitionSelector from "../components/molecules/CognitionSelector";
23. import Slider from "../components/atoms/Slider";
24. import Checkbox from "../components/atoms/Checkbox";
25. import Button from "../components/atoms/Button";
26. import "../styles/Seguimiento.css";
27.
28. /**
29.  * @function Seguimiento
30.  * @description Componente principal de la página de seguimiento. Genera el formulario de seguimiento.
31.  * @returns {JSX.Element} La página de seguimiento.
32.  */
33. const Seguimiento = () => {
34.   const navigate = useNavigate();
35.   const { user } = useAuthStore();
36.   const [currentStep, setCurrentStep] = useState(1);
37.   const [loading, setLoading] = useState(false);
38.   const totalSteps = 8;
39.
40.   // Estado del formulario completo
41.   const [formData, setFormData] = useState({
42.     estadoAnimo: {
43.       emociones: [],
44.       comentario: ''
45.     },
46.     sueno: {
47.       horaInicioSueno: '',
48.       horaDespertar: '',
49.       dificultadDormir: false,
50.       despertaresNocturnos: false,
51.       cansancioDespertar: false,
52.       suenoNoReparador: false,
```

```

53.         sueñosVividos: false,
54.         notasSueno: ''
55.     },
56.     actividadFisica: [],
57.     energia: {
58.         nivel: 'ok',
59.         intensidad: 5
60.     },
61.     alimentacion: {
62.         regularidadComidas: '',
63.         calidadDieta: {
64.             frutasVerduras: false,
65.             ultraprocesados: false,
66.             azucar: false,
67.             cafeina: false,
68.             alcohol: false
69.         },
70.         apetito: 'normal'
71.     },
72.     interaccionesSociales: {
73.         cantidad: 'sociable',
74.         calidad: 'apoyo',
75.         notasSociales: ''
76.     },
77.     cognicion: [],
78.     actividadesPlacenteras: [],
79.     medicacion: []
80. });
81.
82. /**
83.  * @function handleSubmit
84.  * @description Envía los datos del formulario de seguimiento al
85.  * @async
86.  */
87. const handleSubmit = async () => {
88.     setLoading(true);
89.     try {
90.         const { apiFetch } = await import('../config/api');
91.         const response = await apiFetch(apiConfig.endpoints.regi
92.             method: 'POST',
93.             body: JSON.stringify(formData)
94.         });
95.
96.         if (response.ok) {
97.             alert('¡Registro guardado exitosamente!');
98.             navigate('/home');
99.         } else {
100.             const data = await response.json();
101.             alert(`Error: ${data.message} || 'No se pudo guardar
102.         }
103.     } catch (error) {
104.         console.error('Error al guardar el registro:', error);
105.         alert('Error al guardar el registro');
106.     } finally {
107.         setLoading(false);
108.     }
109. };
110.
111. /**
112.  * @function nextStep
113.  * @description Avanza al siguiente paso del formulario.
114.  */
115. const nextStep = () => setCurrentStep(prev => Math.min(prev + 1,
116.

```

```

117.     /**
118.      * @function prevStep
119.      * @description Retrocede al paso anterior del formulario.
120.      */
121.     const prevStep = () => setCurrentStep(prev => Math.max(prev - 1,
122.
123.     /**
124.      * @function getStepLabel
125.      * @description Obtiene la etiqueta para un paso específico del
126.      * @param {number} step - El número del paso.
127.      * @returns {string} La etiqueta del paso.
128.      */
129.     const getStepLabel = (step) => {
130.         const labels = ['Ánimo', 'Sueño', 'Actividad', 'Energía', 'A
131.         return `${step}. ${labels[step - 1]}`;
132.     };
133.
134.     return (
135.         <div className="seguimiento-page">
136.             <div className="seguimiento-container">
137.                 <header className="seguimiento-header">
138.                     <h1>Seguimiento Diario</h1>
139.                     <p>Completa tu registro del día - Paso {currentS
140.                 </header>
141.
142.                 { /* Progress indicator */}
143.                 <div className="seguimiento-progress">
144.                     <div className="seguimiento-progress_bar">
145.                         <div
146.                             className="seguimiento-progress_fill"
147.                             style={{ width: `${(currentStep / totalS
148.                         </div>
149.                     </div>
150.                     <div className="seguimiento-progress_steps">
151.                         {[1, 2, 3, 4, 5, 6, 7, 8].map(step => (
152.                             <span key={step} className={currentStep
153.                                 {getStepLabel(step)}
154.                             </span>
155.                         ))}
156.                     </div>
157.                 </div>
158.
159.                 { /* Step 1: Estado de Ánimo */}
160.                 {currentStep === 1 && (
161.                     <div className="seguimiento-step">
162.                         <EmotionSelector
163.                             emociones={formData.estadoAnimo.emocione
164.                             onChange={(emociones) => setFormData({
165.                                 ...formData,
166.                                 estadoAnimo: { ...formData.estadoAni
167.                             }}}
168.                             comentario={formData.estadoAnimo.comenta
169.                             onComentarioChange={(comentario) => setF
170.                                 ...formData,
171.                                 estadoAnimo: { ...formData.estadoAni
172.                             }}}
173.                         </div>
174.                     </div>
175.                 )}
176.
177.                 { /* Step 2: Sueño */}
178.                 {currentStep === 2 && (
179.                     <div className="seguimiento-step">
180.                         <h3>Sueño</h3>

```

```

181. <p className="step-description">¿Cómo has dormido?</p>
182.
183. <div className="form-group">
184.   <label>Hora de inicio de sueño</label>
185.   <input
186.     type="time"
187.     value={formData.sueno.horaInicioSueno}
188.     onChange={(e) => setFormData({
189.       ...formData,
190.       sueno: { ...formData.sueno, horaInicioSueno: e.target.value }
191.     })}
192.   />
193. </div>
194.
195. <div className="form-group">
196.   <label>Hora de despertar</label>
197.   <input
198.     type="time"
199.     value={formData.sueno.horaDespertar}
200.     onChange={(e) => setFormData({
201.       ...formData,
202.       sueno: { ...formData.sueno, horaDespertar: e.target.value }
203.     })}
204.   />
205. </div>
206.
207. <div className="form-group">
208.   <h4>Características del sueño</h4>
209.   <div className="checkbox-group">
210.     <Checkbox
211.       label="Dificultad para dormir"
212.       checked={formData.sueno.dificultadParaDormir}
213.       onChange={(checked) => setFormData({
214.         ...formData,
215.         sueno: { ...formData.sueno, dificultadParaDormir: checked }
216.       })}
217.     />
218.     <Checkbox
219.       label="Despertares nocturnos"
220.       checked={formData.sueno.despertaresNocturnos}
221.       onChange={(checked) => setFormData({
222.         ...formData,
223.         sueno: { ...formData.sueno, despertaresNocturnos: checked }
224.       })}
225.     />
226.     <Checkbox
227.       label="Cansancio al despertar"
228.       checked={formData.sueno.cansancioAlDespertar}
229.       onChange={(checked) => setFormData({
230.         ...formData,
231.         sueno: { ...formData.sueno, cansancioAlDespertar: checked }
232.       })}
233.     />
234.     <Checkbox
235.       label="Sueño no reparador"
236.       checked={formData.sueno.suenoNoReparador}
237.       onChange={(checked) => setFormData({
238.         ...formData,
239.         sueno: { ...formData.sueno, suenoNoReparador: checked }
240.       })}
241.     />
242.     <Checkbox
243.       label="Sueños vívidos"
244.       checked={formData.sueno.suenosVividos}

```

```

245.             onChange={ (checked) => setFormDa
246.                 ...formData,
247.                 sueno: { ...formData.sueno,
248.                     }})
249.         />
250.     </div>
251. </div>
252.
253.     <div className="form-group">
254.         <label>Notas adicionales</label>
255.         <textarea
256.             value={formData.sueno.notasSueno}
257.             onChange={ (e) => setFormData({
258.                 ...formData,
259.                 sueno: { ...formData.sueno, nota
260.                     }})
261.             placeholder="Algo más sobre tu sueño
262.             rows={3}
263.         />
264.     </div>
265. </div>
266. )}
267.
268. { /* Step 3: Actividad Física */}
269. {currentStep === 3 && (
270.     <div className="seguimiento-step">
271.         <h3>Actividad Física</h3>
272.         <p className="step-description">¿Qué activid
273.
274.         <ActivitySelector
275.             actividades={formData.actividadFisica}
276.             onChange={ (actividades) => setFormData({
277.                 ...formData,
278.                 actividadFisica: actividades
279.             })}
280.         />
281.     </div>
282. )}
283.
284. { /* Step 4: Energía */}
285. {currentStep === 4 && (
286.     <div className="seguimiento-step">
287.         <h3>Energía y Vitalidad</h3>
288.         <p className="step-description">¿Cómo ha sid
289.
290.         <div className="form-group">
291.             <label>Nivel de energía</label>
292.             <div className="energy-selector">
293.                 { ['agotamiento', 'cansancio', 'ok',
294.                     <label key={nivel} className="ra
295.                         <input
296.                             type="radio"
297.                             name="energia"
298.                             value={nivel}
299.                             checked={formData.energi
300.                             onChange={ (e) => setForm
301.                                 ...formData,
302.                                 energia: { ...formDa
303.                                     }})
304.                         />
305.                         <span>{nivel.charAt(0).toUpp
306.                     </label>
307.                 )}
308.             </div>

```

```

309.         </div>
310.
311.         <Slider
312.             label="Intensidad de la energía"
313.             value={formData.energia.intensidad}
314.             onChange={(value) => setFormData({
315.                 ...formData,
316.                 energia: { ...formData.energia, inte
317.             })}
318.             min={1}
319.             max={10}
320.         />
321.     </div>
322. })}
323.
324.     /* Step 5: Alimentación */
325.     {currentStep === 5 && (
326.         <div className="seguimiento-step">
327.             <h3>Alimentación</h3>
328.             <p className="step-description">¿Cómo ha sido
329.
330.             <div className="form-group">
331.                 <label>Regularidad de comidas</label>
332.                 <textarea
333.                     value={formData.alimentacion.regular
334.                     onChange={(e) => setFormData({
335.                         ...formData,
336.                         alimentacion: { ...formData.alim
337.                     })}
338.                     placeholder="Describe cuántas comida
339.                     rows={2}
340.                 />
341.             </div>
342.
343.             <div className="form-group">
344.                 <h4>Calidad de la dieta</h4>
345.                 <div className="checkbox-group">
346.                     <Checkbox
347.                         label="Frutas/Verduras"
348.                         checked={formData.alimentacion.c
349.                         onChange={(checked) => setFormDa
350.                         ...formData,
351.                         alimentacion: {
352.                             ...formData.alimentacion
353.                             calidadDieta: { ...formD
354.                         }
355.                     })}
356.                 />
357.                     <Checkbox
358.                         label="Ultraprocesados"
359.                         checked={formData.alimentacion.c
360.                         onChange={(checked) => setFormDa
361.                         ...formData,
362.                         alimentacion: {
363.                             ...formData.alimentacion
364.                             calidadDieta: { ...formD
365.                         }
366.                     })}
367.                 />
368.                     <Checkbox
369.                         label="Azúcar"
370.                         checked={formData.alimentacion.c
371.                         onChange={(checked) => setFormDa
372.                         ...formData,

```

```

373.             alimentacion: {
374.                 ...formData.alimentacion
375.             },
376.             calidadDieta: { ...formData.calidadDieta }
377.         }
378.     }
379.     <Checkbox
380.         label="Cafeína"
381.         checked={formData.alimentacion.cafeina}
382.         onChange={(checked) => setFormData(
383.             ...formData,
384.             alimentacion: {
385.                 ...formData.alimentacion,
386.                 cafeina: checked ? true : false,
387.             },
388.             calidadDieta: { ...formData.calidadDieta }
389.         )}
390.     <Checkbox
391.         label="Alcohol"
392.         checked={formData.alimentacion.alcohol}
393.         onChange={(checked) => setFormData(
394.             ...formData,
395.             alimentacion: {
396.                 ...formData.alimentacion,
397.                 alcohol: checked ? true : false,
398.             },
399.             calidadDieta: { ...formData.calidadDieta }
400.         )}
401.     </div>
402. </div>
403.
404. <div className="form-group">
405.     <label>Apetito</label>
406.     <div className="energy-selector">
407.         {['disminuido', 'normal', 'aumentado'].map((nivel) => (
408.             <label key={nivel} className="radio-label">
409.                 <input
410.                     type="radio"
411.                     name="apetito"
412.                     value={nivel}
413.                     checked={formData.alimentacion.apetito === nivel}
414.                     onChange={(e) => setFormData(
415.                         ...formData,
416.                         alimentacion: { ...formData.alimentacion,
417.                             apetito: e.target.value }
418.                     )}
419.                 <span>{nivel.charAt(0).toUpperCase}
420.             </label>
421.         ))}
422.     </div>
423. </div>
424. </div>
425. )}
426.
427. /* Step 6: Interacciones Sociales */
428. {currentStep === 6 && (
429.     <div className="seguimiento-step">
430.         <h3>Interacciones Sociales</h3>
431.         <p className="step-description">¿Cómo han sido tus interacciones sociales recientemente?</p>
432.
433.         <div className="form-group">
434.             <label>Cantidad de interacciones</label>
435.             <div className="energy-selector">
436.                 {['sociable', 'introvertido'].map((tipo) => (

```

```

437.         <label key={tipo} className="radio-label">
438.             <input
439.                 type="radio"
440.                 name="cantidad"
441.                 value={tipo}
442.                 checked={formData.interaccionesSociales.cantidad === tipo}
443.                 onChange={(e) => setFormData({
444.                     ...formData,
445.                     interaccionesSociales: {
446.                         ...formData.interaccionesSociales,
447.                         cantidad: e.target.value
448.                     }
449.                 })}
450.             />
451.             <span>{tipo.charAt(0).toUpperCase}
452.         </label>
453.     )})}
454. </div>
455. </div>
456.
457. <div className="form-group">
458.     <label>Calidad de interacciones</label>
459.     <div className="energy-selector">
460.         {['apoyo', 'conflicto'].map(tipo =>
461.             <label key={tipo} className="radio-label">
462.                 <input
463.                     type="radio"
464.                     name="calidad"
465.                     value={tipo}
466.                     checked={formData.interaccionesSociales.calidad === tipo}
467.                     onChange={(e) => setFormData({
468.                         ...formData,
469.                         interaccionesSociales: {
470.                             ...formData.interaccionesSociales,
471.                             calidad: e.target.value
472.                         }
473.                     })}
474.                 />
475.                 <span>{tipo.charAt(0).toUpperCase}
476.             </label>
477.         )})}
478.     </div>
479. </div>
480.
481. <div className="form-group">
482.     <label>Notas sobre interacciones sociales</label>
483.     <textarea
484.         value={formData.interaccionesSociales.notas}
485.         onChange={(e) => setFormData({
486.             ...formData,
487.             interaccionesSociales: {
488.                 ...formData.interaccionesSociales,
489.                 notasSociales: e.target.value
490.             }
491.         })}
492.         placeholder="Describe tus interacciones sociales"
493.         rows={3}
494.     />
495. </div>
496. </div>
497. )}
498.
499. { /* Step 7: Cognición */}
500. {currentStep === 7 && (

```



```

501.         <div className="seguimiento-step">
502.             <CognitionSelector
503.                 cognicion={formData.cognicion}
504.                 onChange={({cognicion}) => setFormData({
505.                     ...formData,
506.                     cognicion
507.                 })}
508.             />
509.         </div>
510.     )}
511.
512.     {/* Step 8: Actividades Placenteras y Medicación */}
513.     {currentStep === 8 && (
514.         <div className="seguimiento-step">
515.             <h3>Actividades Placenteras y Medicación</h3>
516.             <p className="step-description">Información
517.
518.                 <div className="info-box info-box--warning">
519.                     <p> <strong>Actividades Placenteras:</
520.                 </div>
521.
522.                 <div className="info-box info-box--info" sty
523.                     <p> <strong>Medicación:</strong> Podrá
524.                 </div>
525.
526.                 <div className="form-group" style={{ marginT
527.                     <h4>Resumen de tu registro</h4>
528.                     <div className="summary-box">
529.                         <div className="summary-item">
530.                             <span className="summary-label">
531.                                 <span className="summary-value">
532.                             </div>
533.                         <div className="summary-item">
534.                             <span className="summary-label">
535.                                 <span className="summary-value">
536.                         </div>
537.                         <div className="summary-item">
538.                             <span className="summary-label">
539.                                 <span className="summary-value">
540.                         </div>
541.                     </div>
542.                 </div>
543.             </div>
544.         )}
545.
546.     {/* Navigation buttons */}
547.     <div className="seguimiento-actions">
548.         {currentStep > 1 && (
549.             <Button variant="outline" onClick={prevStep}
550.                 ← Anterior
551.             </Button>
552.         )}
553.         {currentStep < totalSteps ? (
554.             <Button variant="primary" onClick={nextStep}
555.                 Siguiente →
556.             </Button>
557.         ) : (
558.             <Button
559.                 variant="primary"
560.                 onClick={handleSubmit}
561.                 disabled={loading}
562.             >
563.                 {loading ? 'Guardando...' : '✓ Guardar P
564.             </Button>

```

```
565.         })
566.     </div>
567. </div>
568. </div>
569. );
570. };
571.
572. export default Seguimiento;
```

Documentation generated by [JSDoc 4.0.5](#) on Thu Dec 11 2025 09:59:03 GMT+0000
(Coordinated Universal Time) using the [docdash](#) theme.